

Öğrenme fazları: **1 • Ön-öğretim** **2 • İçerik** **3 • Katılım** **4 • Değerlendirme** **5 • İzleme**

1 • Ön-öğretim **Ön Bilgi:** Hf.2'nin koltuk-plani/bilet-formu HTML yapısı (statikti) değişme-den korunur; YG I/II'nin C/C++ değer/referans ayrımı ve Liste<T> şablon sınıfı JS/TS karşılıklıla-ryla köprülenir.

» Ünite 1 — JavaScript Dil Modeli: Değer/Referans, Fonksiyon, Closure, Pro-totip

let/const ve Blok Kapsamı

let değeri değiştirilebilir; const bağı değiştirilemez — nesne/diziyse içeriği yine değişebilir. Eski var fonksiyon kapsamlıdır, bu derste let/const (blok kapsamlı) kullanılır.

Değer vs Referans Semantiği

İlkel tipler (number, string, boolean) bir değişkene kopyalanarak ata-nır (C++'ın int b = a; gibi); nesne/dizi her zaman ortak belleğe işa-ret eden bir referans taşır (C++'ın T&'ına benzer) — kopya seçeneğiniz yoktur.

Fonksiyonlar: Bildirim, İfade, Ok Fonksiyonu

Fonksiyon bildirimi her yerden çağrılabilir (*hoisting*); fonksiyon ifade-si yalnız atamadan sonra çağrılabilir. Ok fonksiyonu this'i tanımlan-dığı yerden alır (leksik) — normal fonksiyonun this'i çağrıldığı yere göre değişir.

Closure

Bir iç fonksiyon, tanımlandığı dış fonksiyon kapandıktan sonra bile o kapsamdaki değişkenlere erişmeye devam eder — olusturSayac() her çağrıldığında kendi bağımsız sayaç tutan yeni fonksiyon döndü-rür.

Nesne, Dizi, Dizi Metotları

map dönüştürür, filter koşulu sağlayanları seçer, find ilk eşleşeni döndürür — üçü de orijinali değiştirmez; ... spread sığ bir kopya üre-tir.

Prototip Zinciri ve class

Bir nesnede bulunmayan özellik, prototipinde yukarı doğru aranır. ES6 class bu mekanizmanın üstüne kurulmuş okunaklı bir kısayol-dur — class metotları da prototype üzerindedir.

Ortak Hata / Tuzak

C++'tan gelen alışkanlıkla “diziyi kopyalayarak mı referansla mı geçiririm, ben mi seçerim” diye sorulur. Cevap: seçim hakkı-nız yoktur — JS'te nesne/dizi parametreleri her zaman referans olarak geçer; değiştirmeden bırakmak için önce spread ile kopya oluşturun.

» Ünite 2 — Modüller ve Araç Zinciri: ES Modülleri, npm, Vite

ES Modülü (import/export)

Her .js/.ts dosyası kendi kapsamını korur; yalnızca export ile işa-retlenenler başka dosyalarca import edilebilir.

npm ve package.json

package.json projenin kimliğini, dependencies/ devDependencies paketlerini, scripts kısayollarını tutar.

Vite Proje Kurulumu

npm create vite@latest -- --template vanilla-ts saf TypeScript şablonunu kurar (React şablonu değil) — index.html, src/main.ts, tsconfig.json, package.json oluşur.

dev vs build vs preview

npm run dev anlık (HMR) bir geliştirme sunucusu açar, tam derle-mez, hız önceliklidir. npm run build Önce tsc ile tüm projeyi tip ha-tasına karşı denetler, sonra dist/'e yazar — bir tip hatası bile varsa başarısız olur.

npm create vite@latest -- --template vanilla-ts	Saf TypeScript Vite projesi oluşturur
npm run dev	Anlık (HMR) geliştirme sunucusunu localhost:5173'te açar
npm run build	JS üretmeden tüm projeyi tip hatalarına karşı denetler
npm run preview	tsc denetiminden geçirip optimize edilmiş dist/ üretir

» Ünite 3 — TypeScript Tip Sistemi: Etkinlik, Koltuk, Bilet

Temel Tipler ve Tip Çıkarımı

string, number (int/double ayrımı yok), boolean, T[. any tip dene-timini kapatır (kaçının); unknown daraltma zorunlu kılar. let sayi = 5; yazınca tip otomatik çıkarılır.

interface vs type

İkisi de bir nesnenin şeklini tanımlar. interface extends ile genişleti-lebilir; type union/kesişim için daha esnekler.

Union ve Literal Tipler

'bos' | 'secili' | 'dolu' — bir değişken önceden tanımlı birkaç değerdan birini alabilir (| = “ya da”); üçü birleşip KoltukDurumu bir-leşimini oluşturur.

readonly

readonly id: number; — nesne oluşturulduktan sonra bu alana ya-pılan atama tsc tarafından hata sayılır. Kimlik alanları doğal birer readonly adaydır (kurucuda atayıp const yapmaya benzer).

Generic — Liste<T>

YG I'in Liste<T> şablonuyla aynı fikir: bir tip/fonksiyon hangi veri tiptiyle çalışacağını kullanım anında alır — Liste<Etkinlik> ve Liste<Koltuk> aynı arayüzü kullanır.

Tip Daraltma (narrowing)

typeof, ===, in kontrolünden sonra TypeScript o bloğun içinde değış-kenin hangi alt tipte olduğunu “bilir”.

Ortak Hata / Tuzak

koltuk.durum = 'rezerve' yazmak tsc hatası verir — 'rezerve' union'da tanımlı değil. npx tsc --noEmit bu hatayı çalışmadan önce yakalar; düz JS'te sessizce geçerdi.

» Ünite 4 — Asenkron JS ve DOM

Olay Döngüsü: Çağrı Yığını ve Görev Kuyrukları

JS tek iş parçacığında çalışır (YG II'nin std::thread gerçek paralel-liğinin aksine); çağrı yığını boşalmadan görev kuyruğu işlenmez — mikro kuyruk (Promise) her zaman makro kuyruktan (setTimeout) önce boşaltılır.

Promise ve async/await

Promise, bir asenkron işlemin “şu an sonucu yok ama ileride ya başa-ryla ya hatayla sonuçlanacak” sözüdür (.then()).catch() zinciri). async/await aynı zinciri senkron görünümlü yazar — await yalnızca içinde bulunduğu fonksiyonu duraklatır, tüm programı bloklamaz.

DOM API: Seçme, Dinleme, Oluşturma

querySelector('#id') seçer; addEventListener('click', fn) olay dinler; createElement('div') + appendChild yeni öge ekler.

Ortak Hata / Tuzak

Yeni başlayanlar setTimeout(fn, 0)'ın fn'i hemen çalıştıracağı-nı sanır — oysa 0 yalnızca “en kısa gecikmeyle kuyruğa gir” de-mektir, mevcut senkron kod bitene kadar çalışmaz.

olay_dongusu_sirasi:

```
1 // JS tek iş parçacığında çalışır: çağrı yığını tamamen boşalmadan
   görev
2 // kuyruğundaki hiçbir iş alınmaz. setTimeout(fn, 0) bile "hemen" değil
   '
3 // "sıra gelince" demektir; mikro görev kuyruğu (Promise) makro görev
   // kuyruğundan (setTimeout) ÖNCE boşaltılır.
4 console.log("A"); // senkron
5 setTimeout(() => console.log("B"), 0); // makro görev kuyruğu
6 Promise.resolve().then(() => console.log("C")); // mikro görev kuyruğu
7 console.log("D"); // senkron
8
9
10 // Beklenen sıra: A, D, C, B
```

5 • İzleme **Nerede Kullanılır:** İlkel (kopya) → number/string/boolean · Nesne/dizi (referans) → ... ile kopyala · Genişletilebilir şekil → interface · Sabit kimlik → readonly · Geliştirirken → npm run dev · Teslim öncesi → npm run build + tsc --noEmit